

Section 09 - Deep Dive In Component

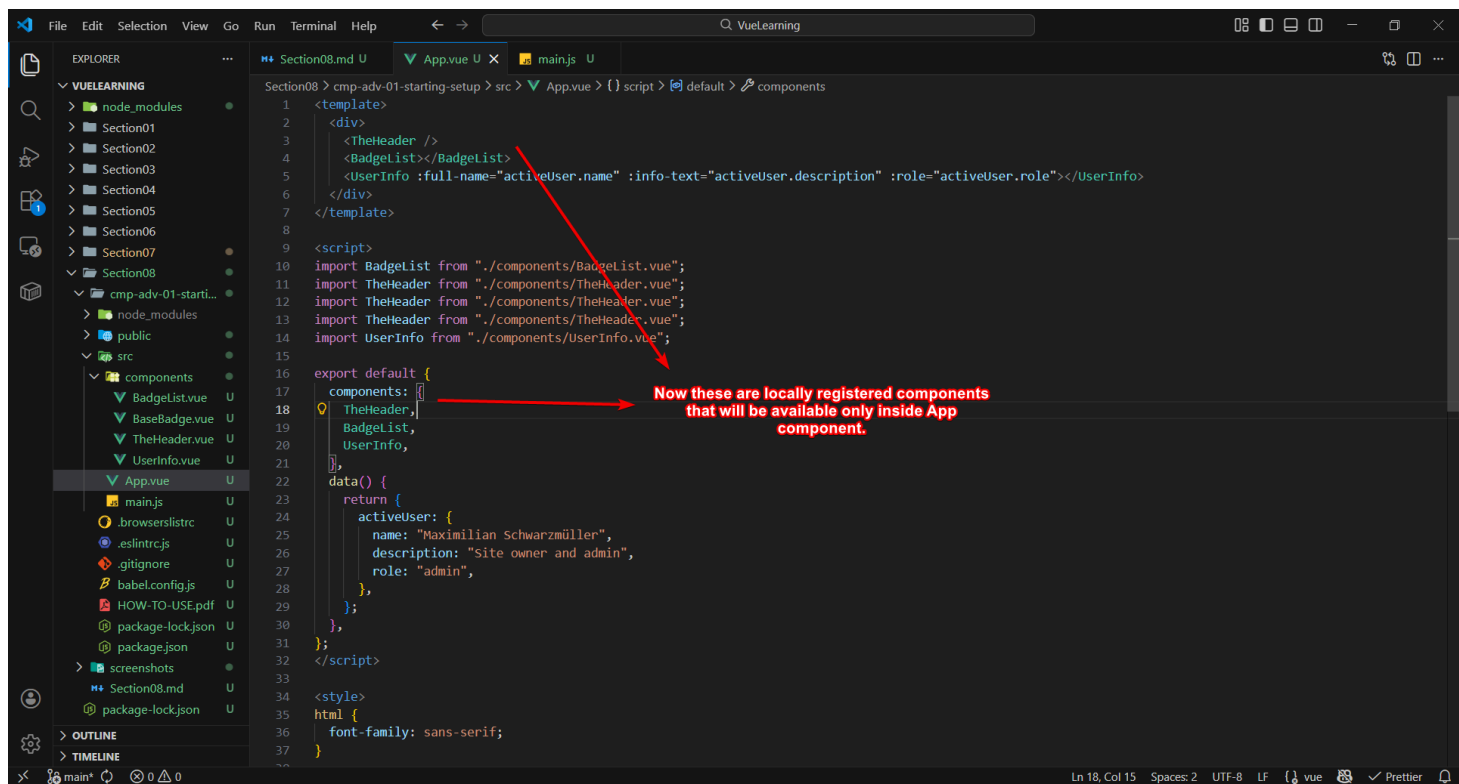
Globally Registered Components

```
1 import { createApp } from 'vue';
2
3 import App from './App.vue';
4 import TheHeader from './components/TheHeader.vue';
5 import BaseBadge from './components/BaseBadge.vue';
6 import BadgeList from './components/BadgeList.vue';
7 import UserInfo from './components/UserInfo.vue';
8
9 const app = createApp(App);
10
11 app.component('the-header', TheHeader);
12 app.component('base-badge', BaseBadge);
13 app.component('badge-list', BadgeList);
14 app.component('user-info', UserInfo);
15
16 app.mount('#app');
```

Here we are registering all components globally that means each one of these components can be used by any one of the components

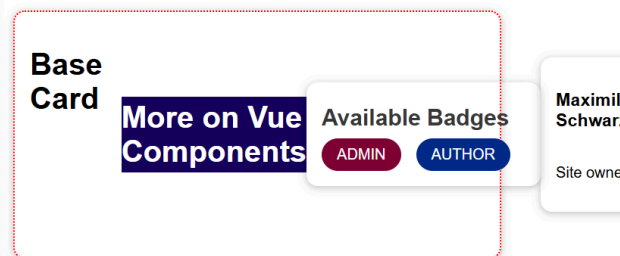
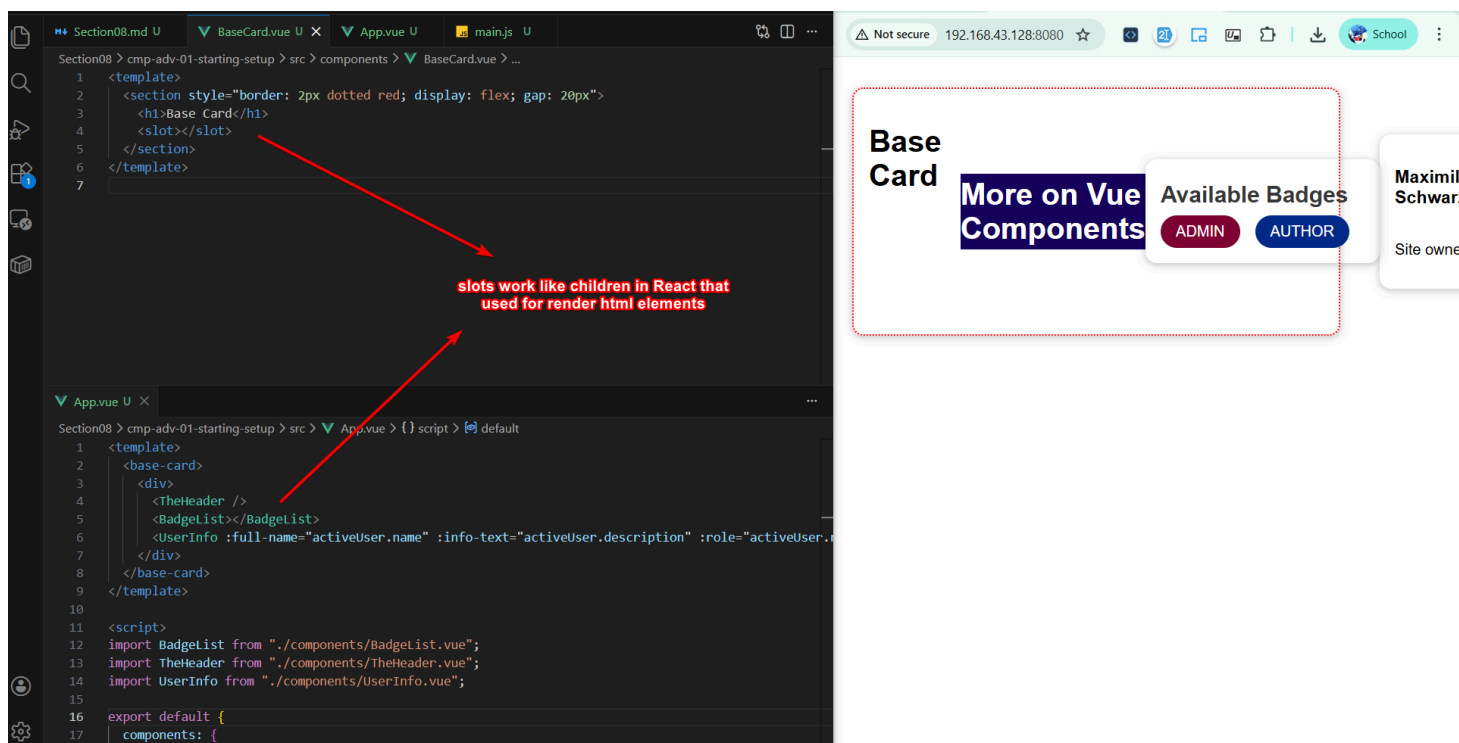
Globally registered components are available globally.

Locally Registered Components



Locally registered components are only available inside the component it is being registered.

Slots *equiv. children* in React



Named Slots

We can use **named slots** as well.

Section08.md U

BaseCard.vue U

App.vue U

main.js U

Section08 > cmp-adv-01-starting-setup > src > App.vue > ...

1 <template>

2 <base-card>

3 <template v-slot:header>

4 <TheHeader />

5 </template>

6 <template v-slot:default>

7 <BadgeList></BadgeList>

8 </template>

9 <template #footer>

10 <UserInfo :full-name="activeUser.name" :info-text="activeUser.description" :role="activeUser.role" />

11 </template>

12 </base-card>

13 </template>

14

15 <script>

16 import BadgeList from "../components/BadgeList.vue";

17 import TheHeader from "../components/TheHeader.vue";

18 import UserInfo from "../components/UserInfo.vue";

19 </script>

to use named slots, we need to wrap child elements under template tag with v-slot:name attribute, v-slot has shorthand as "#"

BaseCard.vue U

Section08 > cmp-adv-01-starting-setup > src > components > BaseCard.vue > {} template > section > footer

1 <template>

2 <section style="border: 2px dotted red">

3 <header>

4 <slot name="header"></slot>

5 </header>

6 <main>

7 <h1>Base Card</h1>

8 <slot></slot>

9 </main>

10 <footer>

11 <slot name="footer"></slot>

12 </footer>

13 </section>

14 </template>

15

We can have named slots but there can be at most one default slot

localhost:8080

More on Vue Components

Base Card

Available Badges

ADMIN AUTHOR

Maximilian Schwarzmüller

ADMIN

Site owner and admin

Scoped Slots

When we want to pass data from component to the component that renders it, we need to use **scoped slots**.

Section08.md U BaseCard.vue U GoalsCard.vue X App.vue U main.js U

```
Section08 > cmp-adv-01-starting-setup > src > components > GoalsCard.vue > {} template > section > ul
1 <template>
2 <section>
3 <ul>
4 <li v-for="goal in this.goals" :key="goal.title">
5 <slot :item="goal" another-prop="..."></slot>
6 </li>
7 </ul>
8 </section>
9 </template>
10
11 <script>
12 export default {
13   data: () => {
14     return {
15       goals: [
16         {
17           title: "Complete Vue Course",
18           description: "Complete Vue Course in 3 weeks"
19         }
20       ]
21     }
22   }
23 }
24 </script>
```

Here we are send two props item and another-prop, that will be available to component that will render it

Scoped slots are used when we want to send data from child component to the component that rendering it.

App.vue U X

```
Section08 > cmp-adv-01-starting-setup > src > App.vue > {} template > GoalsCard
1 <template>
2 <base-card>
3 <template #footer>
4
5 </template>
6 </base-card>
7 <GoalsCard v-slot="slotProps">
8 <h2>{{ slotProps.item.title }}</h2>
9 <h3>{{ slotProps.item.description }}</h3>
10 <h3>{{ slotProps["another-prop"] }}</h3>
11 </GoalsCard>
12 </template>
13
14 <script>
15 import BadgeList from "../components/BadgeList.vue";
16 import TheHeader from "../components/TheHeader.vue";
17 import UserInfo from "../components/UserInfo.vue";
18 import GoalsCard from "../components/GoalsCard.vue";
19
20 export default {
21   components: {
22     BadgeList,
23     TheHeader,
24     UserInfo,
25     GoalsCard
26   },
27   data() {
28     return {
29       selectedComponent: "active-goals"
30     }
31   },
32   methods: {
33     setSelectedComponents(cmp) {
34       this.selectedComponent = cmp;
35     }
36   }
37 }
38 </script>
```

When rendering scoped slots, we can use v-slot="AnyName" and all props we will available inside this AnyName object.

More on Vue Components

Base Card

Available Badges

ADMIN AUTHOR

Maximilian Schwarzmüller ADMIN

Site owner and admin

Complete Vue Course

Create Vue Projects

Complete Two Vue Projects in 1 week

Complete Vue Course in 2 weeks

Dynamic Components

Vue JS provide us `component` tag that accepts `is` attribute to render component whose name is provided.

Section08.md U BaseCard.vue U GoalsCard.vue U App.vue U ActiveGoals.vue U ManageGoals.vue

```
Section08 > cmp-adv-01-starting-setup > src > App.vue > {} template > component
1 <!-- <base-card> -->
2
3 <button @click="setSelectedComponents('active-goals')">Active Goals</button>
4 <button @click="setSelectedComponents('manage-goals')">Manage Goals</button>
5
6 <component :is="selectedComponent"></component>
7 </template>
8
9 <script>
10 import ActiveGoals from "../components/ActiveGoals.vue";
11 import ManageGoals from "../components/ManageGoals.vue";
12
13 // import BadgeList from "../components/BadgeList.vue";
14 // import TheHeader from "../components/TheHeader.vue";
15 // import UserInfo from "../components/UserInfo.vue";
16 // import GoalsCard from "../components/GoalsCard.vue";
17
18 export default {
19   components: {
20     ActiveGoals,
21     ManageGoals,
22   },
23   data() {
24     return {
25       activeUser: {
26         name: "Maximilian Schwarzmüller",
27         avatar: "https://i.pravatar.cc/150?img=31",
28       },
29       selectedComponent: "active-goals",
30     };
31   },
32   methods: {
33     setSelectedComponents(cmp) {
34       this.selectedComponent = cmp;
35     }
36   }
37 }
38 </script>
```

We can select component based on condition (aka dynamic component) using Vue JS provided component tag that accepts 'is' attribute and it renders component whose name is provided in this attribute

Active Goals Manage Goals

Active Goals

Feature flag `__VUE_PROD_HYDRATION_MISMATCH_DETAILS__` is not explicitly defined. You are running the esm-bundler build of Vue, which expects these compile-time feature flags to be globally injected via the bundler config in order to get better tree-shaking in the production bundle.

For more details, see <https://link.vuejs.org/feature-flags>.

{results: [-]}

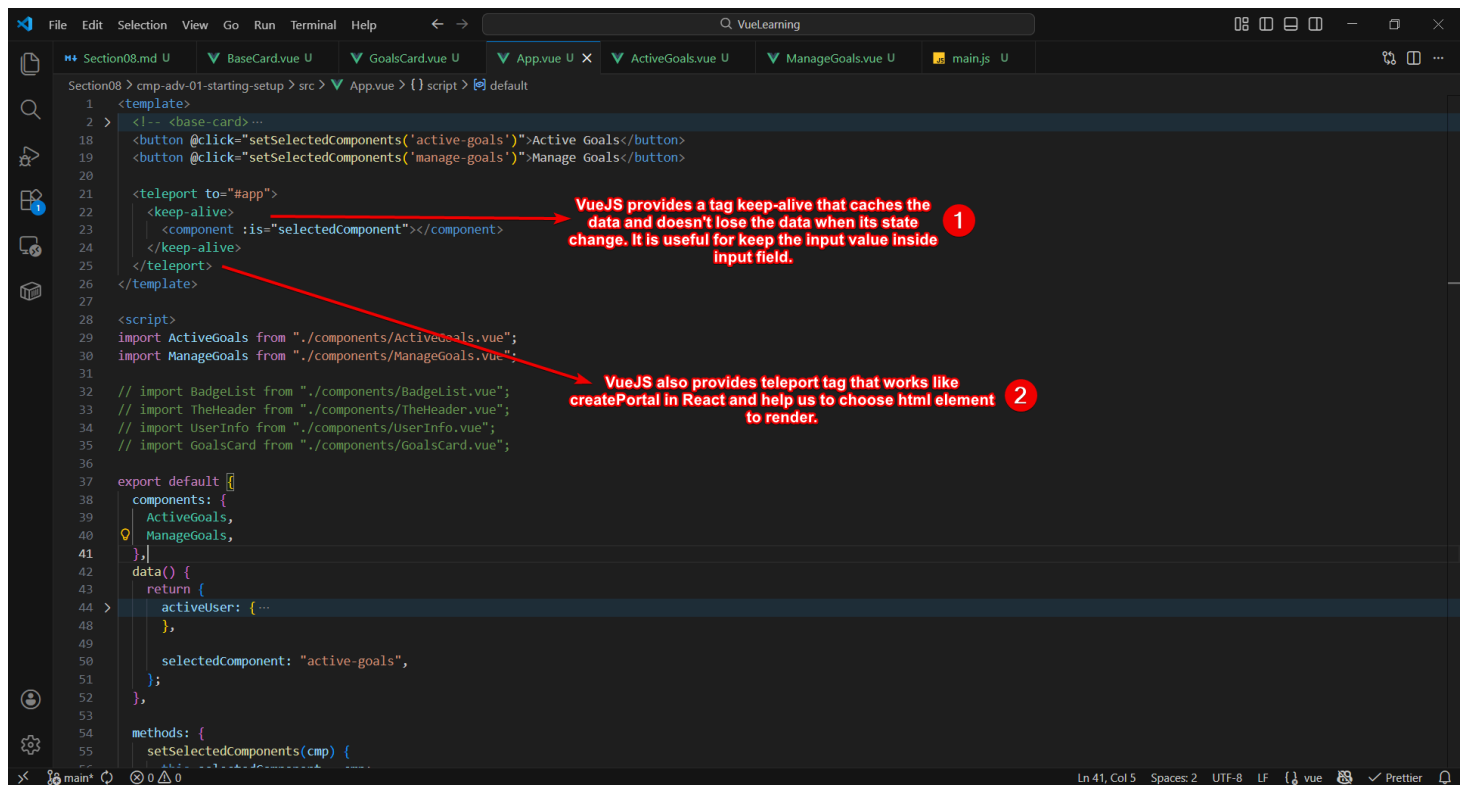
reload

content.js:1

App.vue:15

Keep-Alive and Teleport

Keep-alive tag caches the data and prevent losing data when state of component changes. **Teleport** allows us to choose html element to render the element.



The screenshot shows a VS Code editor with a file named `App.vue` open. The file contains the following code:

```
1 <template>
2   <!-- <base-card> ...
18   <button @click="setSelectedComponents('active-goals')">Active Goals</button>
19   <button @click="setSelectedComponents('manage-goals')">Manage Goals</button>
20
21   <teleport to="#app">
22     <keep-alive>
23       <component :is="selectedComponent"></component>
24     </keep-alive>
25   </teleport>
26 </template>
27
28 <script>
29 import ActiveGoals from "../components/ActiveGoals.vue";
30 import ManageGoals from "../components/ManageGoals.vue";
31
32 // import BadgeList from "../components/BadgeList.vue";
33 // import TheHeader from "../components/TheHeader.vue";
34 // import UserInfo from "../components/UserInfo.vue";
35 // import GoalsCard from "../components/GoalsCard.vue";
36
37 export default {
38   components: {
39     ActiveGoals,
40     ManageGoals,
41   },
42   data() {
43     return {
44       activeUser: { ...
45     },
46     selectedComponent: "active-goals",
47   },
48   methods: {
49     setSelectedComponents(cmp) {
50       // ...
51     }
52   }
53 }
54 </script>
```

Two red arrows point from text annotations to the code:

- Arrow 1 points to the `<keep-alive>` tag (lines 22-24). The annotation reads: "VueJS provides a tag keep-alive that caches the data and doesn't lose the data when its state change. It is useful for keep the input value inside input field." (1)
- Arrow 2 points to the `<teleport to="#app">` tag (line 21). The annotation reads: "VueJS also provides teleport tag that works like createPortal in React and help us to choose html element to render." (2)

Section 09 - Summary

Component Deep Dive

Component Registration & Styles

Components can be registered **globally or locally**: **Prefer local** registration.

Styles can be **global or scoped** to a component: **Prefer scoped** for most components.

Dynamic Components

Components can be **swapped dynamically** via the built-in "component" component.

Component caching can be added via the "keep-alive" component.

Slots

Slots can be used to add a "placeholder" for dynamic HTML code.

Multiple, named slots are possible, **default fallbacks** can be provided. **Scoped slots** allow adv. use-cases.

Teleport & Style Guide

DOM structure can be manipulated via "teleport" – it **keeps the component hierarchy!**

Consider following the official style guide to keep your code clean and understandable.